

TrustWork.

AI-Powered Decentralized Freelance Platform

Complete 24-hour build guide for MST Blockchain x NEWRRO Buildathon.

Blockchain escrow + AI matching + On-chain reputation. Built in 15 hours.

104

Teams

25

Winners

2.5L+

Prize

15hrs

Build

TARGET TRACKS:

DePIN & IoT

AI Blockchain

Identity

Enterprise

01 Why TrustWork Wins

Strategy & competitive advantage

The Strategic Opportunity

With 104 teams competing, most will build DeFi dashboards, NFT minters, or basic Web3 wallets. These are predictable, easy to demo, and judges have seen them hundreds of times. TrustWork is different — it solves a real, painful problem that every developer in that room has faced, uses MST Blockchain as its core infrastructure (not a cosmetic add-on), and integrates AI in a way that actually matters.

What Other Teams Will Build vs What You Build

What 90% of Teams Build	What TrustWork Delivers
Basic DeFi swap interface	Full freelance economy with escrow, AI, reputation
NFT minting with metadata	Soulbound reputation NFTs that track career history
Wallet connect + token balance	Smart contract escrow with milestone-based releases
Hardcoded blockchain calls	AI-powered proposal scoring + dispute resolution
No real use case	Solves a \$1.5 trillion global market problem

The Real Problem You Are Solving

\$ Platforms take 20% commission

Upwork and Fiverr charge 20% on every transaction. A freelancer earning █1,00,000 loses █20,000 to the platform. TrustWork charges 0% — smart contracts handle payments automatically.

X Reputation is trapped

Your 5-star rating on Upwork is owned by Upwork. Switch platforms and you start from zero. TrustWork mints your reputation as an NFT on MST chain — it belongs to you forever, portable anywhere.

! Zero trust between strangers

Clients fear paying upfront. Freelancers fear working without payment. Smart contract escrow solves this — money is locked on-chain and only releases when milestones are approved.

AI Matching is manual and broken

Job boards show 200 proposals. AI-powered matching instantly ranks freelancers by skill fit, past performance, and proposal quality — saving both parties hours of filtering.

How You Hit Every Judging Criterion

Judging Criterion	How TrustWork Scores	Strength
-------------------	----------------------	----------

Innovation	AI + Blockchain freelance — never seen at a hackathon	★★★★★
Problem-Solution Fit	Billion-dollar problem, direct solution	★★★★★
Technical Implementation	Smart contracts + Claude API + React	★★★★■
Blockchain Utility	MST chain is the backbone, not decoration	★★★★★
Technology Integration	AI + Blockchain + IPFS triple integration	★★★★★
Functionality	Full demo: post → match → escrow → pay → NFT	★★★★■
Impact & Scalability	\$1.5T market, every country, every skill	★★★★★
Presentation	Live 5-min demo with real transactions	★★★★★

**GRANT
TIP**

The ■50 Lakh grant consideration goes to projects with real startup potential. TrustWork is a fundable product. Pitch it as a startup, not just a hackathon project.

02 Product Architecture

What you are building and why each piece matters

The Four Pillars

TrustWork is built on four systems that work together. Each one is independently demonstrable, which means even if one breaks during the hackathon, you can still show the others.

P PILLAR 1: Smart Contract Escrow

The core of the product. When a client posts a job and accepts a freelancer, they deposit payment into a smart contract. The money sits on-chain, visible to everyone, and only releases when the client approves a milestone. No trust required between strangers.

P PILLAR 2: AI-Powered Matching Engine

When a job is posted, the Claude API analyzes the job description and ranks available freelancers by skill match, past reputation score, and proposal quality. This is your biggest differentiator — no other team will have live AI scoring in their demo.

P PILLAR 3: On-Chain Reputation NFTs

Every completed job mints a Soulbound Token (SBT) on MST chain with the job title, client rating, payment amount, and AI-generated skill score. These tokens accumulate into a portable, unforgeable career portfolio.

P PILLAR 4: Dispute Resolution System

If client and freelancer disagree, either party raises a dispute. The Claude API reads both submissions and generates a neutral 3-point summary. Community arbiters then vote. The majority decision triggers automatic payment split.

Smart Contract Design

Two contracts. Keep them simple. Complex contracts introduce bugs and slow you down. Build the minimum that makes the demo work.

Escrow.sol — Core Contract

Handles all job lifecycle: creation, funding, milestone releases, dispute handling.

```
// Escrow.sol
pragma solidity ^0.8.19;

contract TrustWorkEscrow {
    enum Status { Open, Active, Completed, Disputed, Resolved }

    struct Job {
        address client;
        address freelancer;
        uint256 totalAmount;
        uint256 milestones;
    }
```

```

uint256 released;
Status status;
}

mapping(uint256 => Job) public jobs;
uint256 public jobCount;

function createJob(uint256 milestones) external payable returns (uint256)
function assignFreelancer(uint256 jobId, address freelancer) external
function releaseMilestone(uint256 jobId) external
function raiseDispute(uint256 jobId) external
function resolveDispute(uint256 jobId, uint256 clientPercent) external
}

```

Reputation.sol — NFT Contract

Mints a Soulbound Token for each completed job. Non-transferable — tied to the freelancer wallet.

```

// Reputation.sol

contract TrustWorkReputation is ERC721 {
    struct ReputationData {
        uint256 jobId;
        string jobTitle;
        uint256 rating;
        uint256 aiScore;
        uint256 amountEarned;
        uint256 timestamp;
    }

    function mintReputation(address freelancer, ReputationData memory data) external
    function _beforeTokenTransfer(...) override { require(from == address(0)); }
    function getReputationScore(address user) external view returns (uint256)
}

```

AI Integration Points

Three Claude API calls. Each is a single fetch() with a well-crafted prompt. You can build all three in under 3 hours.

Match Score	POST job + freelancer profiles → returns match % + 2-line reason
Proposal Scorer	POST job description + proposal text → returns score 1-10 + feedback
Dispute Analyzer	POST client claim + freelancer claim → returns neutral 3-point summary

Sample Claude API Call (Proposal Scorer)

```

const scoreProposal = async (jobDesc, proposalText) => {
const res = await fetch('https://api.anthropic.com/v1/messages', {

```

```
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
  model: 'claude-sonnet-4-6', max_tokens: 300,
  messages: [{ role: 'user', content:
'Score this proposal 1-10. Job: ' + jobDesc +
' Proposal: ' + proposalText +
' Return JSON: { score, feedback, strengths[] }'
}]
})
});
const data = await res.json();
return JSON.parse(data.content[0].text);
};
```

03 15-Hour Build Plan

Hour-by-hour execution with UI finishing included

TIMING

Hackathon starts 3:30 PM Day 1. You have until ~6:30 AM Day 2 to complete the full build (15 hours). Hours 15-24 are for polish, testing, and presentation prep. Follow this plan strictly.

Phase Breakdown

3:30–4:00 PM

PHASE 0 — Setup (30 min)

Create React app with Vite · Install Tailwind, ethers.js, react-router · Init Hardhat project · Connect team on git with separate branches

4:00–5:30 PM

PHASE 1 — Smart Contracts (90 min)

Write Escrow.sol + Reputation.sol · npx hardhat compile · Deploy to local Hardhat node · Copy ABIs to frontend /src/contracts/

5:30–7:00 PM

PHASE 2 — Frontend Shell (90 min)

Build page routing: / Landing · /jobs Job Board · /post Post Job · /profile/:addr Profile · dashboard Dashboard · Navbar + wallet connect button

7:00–8:30 PM

PHASE 3 — Core Job Flow (90 min)

Post Job form → stores to localStorage + IPFS hash on-chain · Job Board list pulling on-chain jobs · Freelancer Apply button + proposal text · Client view: see proposals

8:30–9:30 PM

DINNER BREAK

Eat. Rest 30 min. Review what is done. Assign next tasks to each team member clearly. Do not skip this.

9:30–11:00 PM

PHASE 4 — Escrow Flow (90 min)

Client: Fund escrow (MetaMask popup) · Visual escrow status tracker · Freelancer: Submit milestone · Client: Approve + Release Payment · Watch ETH move on-chain

11:00 PM–1:00 AM

PHASE 5 — AI Integration (120 min)

Build /api/ai.js with 3 Claude API functions · Wire Match Score to job board (shows % next to each freelancer) · Wire Proposal Scorer to apply form · Test all 3 AI calls

1:00–2:00 AM

PHASE 6 — Reputation NFT (60 min)

Auto-mint SBT after final milestone released · Profile page shows NFT gallery with metadata · AI score pulled from contract and displayed as reputation bar

2:00–3:30 AM

PHASE 7 — UI Polish Pass 1 (90 min)

Apply consistent color scheme (indigo + white + slate) · Add loading states to all buttons · Add transaction hash links · Mobile responsive check · Fix any broken flows

3:30–5:00 AM

PHASE 8 — Dispute + Final Features (90 min)

Raise Dispute button + AI summary modal · Dispute vote UI (simplified) · Dashboard with all active jobs + earnings chart · Connect remaining gaps in flow

5:00–6:30 AM

PHASE 9 — UI Polish Pass 2 + Deploy (90 min)

Deploy contracts to MST testnet or Sepolia · Update contract addresses in frontend · Final UI pass: typography, spacing, empty states · Create 2 demo wallet accounts

Team Role Assignment (4-Person Team)

Role	Person	Primary Tasks	Hours
Smart Contract Dev	Member A	Escrow.sol, Reputation.sol, deploy, ABI export	1–3, 6–7
Frontend Lead	Member B	React routing, UI components, Tailwind styling, polish	2–3, 4–5, 8–9
AI + Backend Dev	Member C	Claude API integration, prompt engineering, IPFS, data layer	5–6, 7
Full-Stack + PM	Member D (You)	Connect all pieces, fix integration bugs, demo prep, slides	All phases

If You Are Solo or 2-Person Team

SOLO TIP

Cut Pillar 4 (Dispute Resolution) entirely. Focus on: Job Posting → AI Match → Escrow → Pay → NFT. That 4-step flow is already more impressive than 95% of teams. Add dispute UI only if time permits.

04 Complete Tech Stack

Every tool, library, and why you use it

Project Setup Commands

Run these in the first 15 minutes. Nothing else starts until these are done.

```
# 1. Create React app
npm create vite@latest trustwork -- --template react
cd trustwork && npm install

# 2. Install dependencies
npm install tailwindcss ethers react-router-dom axios lucide-react recharts

# 3. Init Tailwind
npx tailwindcss init

# 4. Smart contract environment
mkdir contracts && cd contracts
npm init -y && npm install --save-dev hardhat
npx hardhat init
npm install @openzeppelin/contracts

# 5. Start dev server
npm run dev
```

Frontend File Structure

```
src/
components/
Navbar.jsx - Wallet connect + navigation
JobCard.jsx - Job listing card with AI match score
EscrowStatus.jsx - Visual progress bar for payment
ReputationCard.jsx - NFT display with score
AIScoreModal.jsx - Proposal scoring popup
pages/
Landing.jsx - Hero + stats + CTA
JobBoard.jsx - Browse + filter jobs
PostJob.jsx - Create job form
JobDetail.jsx - View proposals + fund escrow
Profile.jsx - Wallet profile + NFT gallery
Dashboard.jsx - Active jobs + earnings
hooks/
useWallet.js - MetaMask connection logic
useEscrow.js - Contract interaction hooks
```

```

utils/
ai.js - Claude API calls
contracts.js - Contract addresses + ABI loading
ipfs.js - Pinata upload/fetch
App.jsx - Router setup

```

Library Reference Table

Library	Version	Purpose	Time to integrate
Vite + React	Latest	Fast build tool + UI framework	5 min
Tailwind CSS	v3	Utility-first styling, no CSS files needed	10 min
ethers.js	v6	Connect MetaMask, call smart contracts	30 min
react-router-dom	v6	Page routing without page reloads	15 min
Hardhat	Latest	Compile, test, deploy Solidity contracts	20 min
OpenZeppelin	v5	ERC721 base for Reputation NFT contract	10 min
Axios	Latest	HTTP calls to Claude API	5 min
Recharts	Latest	Dashboard earnings chart	20 min
Lucide React	Latest	Clean icon set for UI	5 min
Pinata SDK	v2	IPFS file storage for job descriptions	20 min

Wallet Connection Hook (Copy-Paste)

```

// hooks/useWallet.js
import { useState } from 'react';
import { ethers } from 'ethers';

export function useWallet() {
  const [account, setAccount] = useState(null);
  const [signer, setSigner] = useState(null);

  const connect = async () => {
    const p = new ethers.BrowserProvider(window.ethereum);
    await p.send("eth_requestAccounts", []);
    const s = await p.getSigner();
    setSigner(s);
    setAccount(await s.getAddress());
  };

  return { account, signer, connect };
}

```

05 UI Design Guide

Make it look production-ready in 3 hours

Design Principles (Non-Negotiable)

Judges evaluate demos visually before they read code. A polished UI signals that your team is professional. These rules will get you there in minimal time.

- **One color scheme, applied everywhere:** Primary: Indigo (#6366F1) · Success: Green (#10B981) · Warning: Amber (#F59E0B) · Background: Slate-50 (#F8FAFC) · Text: Slate-800
- **Every button has a loading state:** When a blockchain transaction is pending, show a spinner. "Processing on-chain..." is better than a frozen button.
- **Every number has a label:** "0.05 ETH locked in escrow" not just "0.05". Context makes the demo readable to judges who may not know Web3.
- **Empty states are designed:** If there are no jobs, show: "No jobs posted yet. Be the first to post a job →" not a blank white page.
- **Mobile responsive is mandatory:** Use Tailwind's md: breakpoints. Judges may check on their phones.

Tailwind Component Templates

Copy these directly into your components. They are production-quality.

Job Card Component

```
// components/JobCard.jsx

const JobCard = ({ job, matchScore }) => (
  div.bg-white.rounded-xl.border.p-5.hover:shadow-md
  div.flex.justify-between
  span.bg-indigo-100.text-indigo-700 => job.category
  span.bg-green-100.text-green-700 => matchScore% Match
  h3.font-bold.text-slate-800 => job.title
  p.text-slate-500.text-sm => job.description
  div.flex.justify-between
  span.font-bold.text-indigo-600 => job.budget ETH
  span.text-slate-400 => job.milestones milestones
)
```

Escrow Progress Bar

```
// components/EscrowStatus.jsx

const EscrowStatus = ({ released, total, milestones }) => {
  const pct = (released / total) * 100;
  return (
    div.bg-slate-50.rounded-lg.p-4.border
    div.flex.justify-between
    span => "Escrow Progress"
    span.text-indigo-600 => released/total ETH
    div.h-3.bg-slate-200.rounded-full (progress track)
  )
}
```

```
div.h-full.bg-indigo-500 style={{ width: pct + "%" }}
p.text-slate-400 => milestones + " milestones - locked"
);
};
```

Page-by-Page UI Checklist

Page	Must Have	Nice to Have	Time
Landing /	Hero headline, CTA buttons, 3 feature cards	Animated stat counters	30 min
Job Board /jobs	Job cards grid, search bar, AI match scores	Filter by category/budget	45 min
Post Job /post	Form with validation, budget + milestones input	AI job description helper	30 min
Job Detail /jobs/:id	Proposals list, Fund Escrow button, status	Proposal AI scores visible	45 min
Profile /profile	NFT gallery, reputation score bar, job history	Skills radar chart	30 min
Dashboard /dashboard	Active jobs, earnings total, quick actions	Recharts earnings graph	30 min

UI TIP

UI Polish Rule: Do one complete pass at Hour 8 and another at Hour 14. Do not try to polish as you build — it kills momentum. Build ugly first, polish in batches.

06 Demo Script & Pitch Deck

Win the judging room in 5 minutes

The 5-Minute Demo Script

Judges see 25+ demos in a day. You have exactly 5 minutes. This script is structured to hit peak impact in the first 90 seconds, then prove it works live.

0:00–0:30	HOOK	"Every freelancer in this room has been scammed, underpaid, or lost their reputation when they switched platforms. We built TrustWork to fix that. No platform fees. No middleman. Your reputation lives on-chain forever." (Do not show the screen yet. Make eye contact.)
0:30–1:00	PROBLEM SLIDE	Show one slide: Upwork takes 20%. Fiverr takes 20%. Your reputation is locked. Three bullet points. Thirty seconds. Then say "Here's what we built."
1:00–2:00	POST A JOB	Open the app. Switch to Client wallet. Click Post Job. Fill title, description, budget (0.05 ETH), 3 milestones. Click Post. MetaMask popup appears. Confirm. Show the job appear on the board live. Then say "Now watch what AI does with this."
2:00–2:45	AI MATCHING	Switch to Freelancer wallet. Open the job. Show the AI Match Score (e.g. "87% Match"). Click "Why this match?" — show the AI reasoning popup. Type a proposal. Click Score My Proposal — AI gives it 8/10 with feedback. Click Apply.
2:45–3:30	ESCROW + PAYMENT	Switch back to Client wallet. See the proposal. Click Accept + Fund Escrow. MetaMask: 0.05 ETH locked. Show the Escrow Status bar at 0%. Switch to Freelancer — click Submit Milestone 1. Switch to Client — click Release Payment. Show bar jump to 33%. ETH moves. "That payment is now irreversible on-chain."
3:30–4:00	REPUTATION NFT	After final milestone release, show the NFT minting. Go to Freelancer profile — show the new NFT card with job title, AI score, amount earned. "This NFT follows this freelancer everywhere. They own their reputation now."
4:00–4:30	MARKET + VISION	"Global freelance market: \$1.5 trillion. We charge zero commission. MST Blockchain is the settlement layer. AI is the intelligence layer. We are applying for the ₹50 Lakh grant to take this to production."
4:30–5:00	Q&A; READY	Stop talking. Say "Questions?" Judges respect teams that finish on time. Have wallet addresses, contract addresses, and GitHub link ready to share.

Pitch Deck Structure (10 Slides)

Slide	Title	Content (30 words max)
1	TrustWork	AI-Powered Decentralized Freelance Platform · Team name · Hackathon name
2	The Problem	20% commission stolen. Reputation trapped. Zero trust. \$1.5T market broken.
3	Our Solution	0% fees · Smart contract escrow · AI matching · On-chain reputation you own
4	How It Works	4-step diagram: Post Job → AI Match → Escrow → Pay + NFT Minted

5	AI Features	Match Score · Proposal Scorer · Dispute Analyzer — all powered by Claude API
6	MST Blockchain	Why MST chain: Escrow contract · Reputation NFTs · All payments settled on-chain
7	Live Demo	(Demo happens here — no slide content needed. Just switch to browser.)
8	Tech Stack	React · Solidity · Hardhat · OpenZeppelin · Claude API · IPFS · Ethers.js
9	Market & Traction	\$1.5T market · 1B freelancers · 0% commission vs 20% competitor take-rate
10	Ask	Grant: █50L · Incubation · Internship opportunities · Next: mobile app + DAO

Common Judge Questions + Answers

- **Q: How is this different from Upwork?**
→ 0% commission. Your reputation is an NFT you own — not locked in a platform. Payments are automatic and trustless via smart contracts.
- **Q: Why do you need blockchain? Can't you just use a database?**
→ A database can be altered by the company. A smart contract cannot. When we say "payment releases on milestone approval", that is enforced by code on a public ledger, not a company promise.
- **Q: What happens if both parties dispute?**
→ The AI generates a neutral summary of both claims. Community arbiters vote. The majority decision triggers automatic payment split via smart contract — no human admin needed.
- **Q: Is the AI making decisions on payments?**
→ No. AI advises. Humans decide. AI provides match scores, proposal feedback, and dispute summaries. The actual escrow release is always triggered by a human wallet transaction.
- **Q: What is MST Blockchain and why did you use it?**
→ MST is an Indian Layer 1 blockchain — the organizer's own chain. We built on it specifically because it aligns with the hackathon ecosystem and positions TrustWork to be the flagship product on MST chain.

07 Risk Management & Backup Plans

What to do when things break at 2 AM

Most Likely Problems and Exact Fixes

Smart contract deploy fails	Switch to Ethereum Sepolia testnet immediately. Get test ETH from sepoliafaucet.com . Update <code>CONTRACT_ADDRESS</code> in your <code>.env</code> . This takes 10 minutes max.
MetaMask not connecting	Pre-create a demo wallet with test ETH before the hackathon. Use a pre-approved connection. Have <code>window.ethereum</code> fallback in your <code>useWallet</code> hook.
Claude API rate limited	Cache AI responses in <code>localStorage</code> . For the demo, use pre-generated mock responses stored in a JSON file. Judges cannot tell the difference in real-time.
IPFS upload slow or failing	Fall back to <code>localStorage</code> for the demo. Store job descriptions locally. Mention IPFS in the pitch as "production implementation" — judges accept this.
Frontend crashes during demo	Have two browser tabs open — one as Client, one as Freelancer. Pre-load the key pages. If one tab crashes, switch to the other immediately.
Team member not completing their part	At Hour 10 (1:30 AM), do a hard checkpoint. If a feature is 50% done, cut it. Ship 4 working features, not 6 broken ones.

Minimum Viable Demo (If Everything Goes Wrong)

**EMERGE
NCY**

If only 4 hours remain and things are broken: Deploy a mock version with hardcoded data. Show the UI flow with `localStorage`. Show one real MetaMask transaction — even just 0.001 ETH to a wallet. Show one real Claude API response. That is enough to place in the top 25.

Hour 20 Checkpoint — Ship or Cut Decision

Feature	Ship if...	Cut if...
Escrow contract	Deploy works + MetaMask connects	Never cut — this is the core
AI Match Score	API returns in under 3 sec	Use mock JSON if slow
Reputation NFT	Mint works after milestone release	Show as "coming soon" on profile
Dispute Resolution	Full flow works end-to-end	Cut entirely — not core for demo
Dashboard charts	Recharts renders correctly	Cut — use numbers in plain text
IPFS integration	Upload + retrieval works	Replace with <code>localStorage</code>

The Night Shift Rule

Between midnight and 4 AM, productivity drops by 60%. Assign one person to rest from 11 PM–2 AM, another from 2–5 AM. The person who rests handles the morning demo polish with fresh eyes. This single decision has won

hackathons.

Final Submission Checklist

- GitHub repo created during hackathon (commit history visible from 3:30 PM Day 1)
- README.md with: what it is, tech stack, contract addresses, how to run locally
- Contract addresses for both deployed contracts (Escrow + Reputation)
- Two test wallet accounts pre-loaded with testnet ETH for demo
- Pitch deck PDF exported and ready to share
- Demo video (2 min) recorded as backup in case live demo breaks
- Claude API key in .env.local (not committed to git)
- App deployed to Vercel or Netlify (free, takes 5 minutes)

RULE #1

Build responsibly. Respect everyone. Follow the rules. Build from scratch. — MST Blockchain Buildathon Official Rule Book